

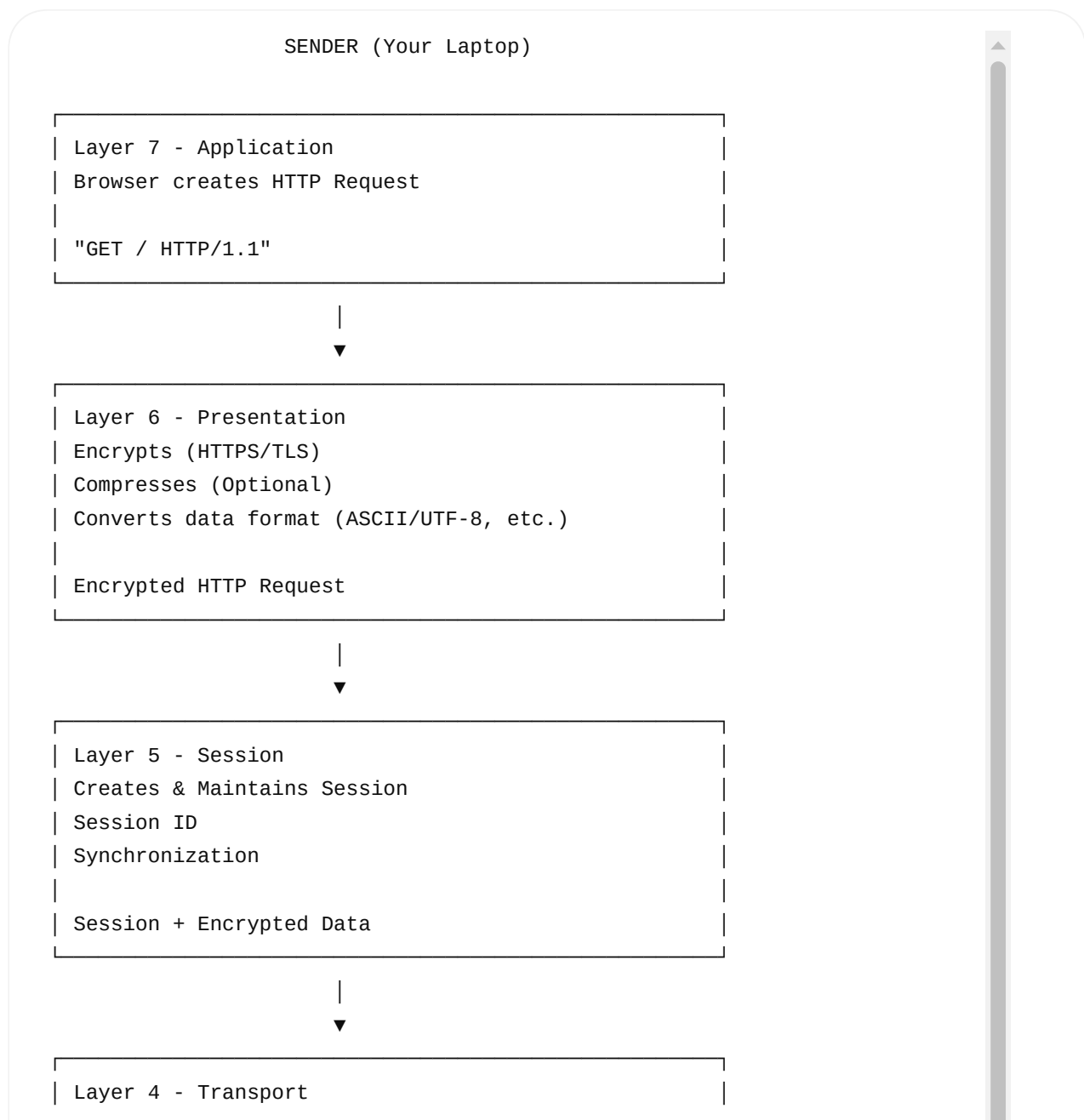
/visualized diagram. provide the seven layer with this usage

Perfect. This is exactly how networking should be learned. Instead of memorizing the OSI layers, **visualize what each layer actually does to your data.**

Let's use a simple example.

You type `https://www.google.com` in your browser and press **Enter**.

OSI Model - Visualized Data Flow



| Adds TCP Header
|
| Source Port : 52000
| Destination : 443
| Sequence Number
| ACK Number
| Window Size
|
| TCP Segment



| Layer 3 - Network
| Adds IP Header
|
| Source IP : 192.168.1.10
| Destination IP : 142.250.xxx.xxx
| TTL
| Protocol = TCP
|
| IP Packet



| Layer 2 - Data Link
| Adds Ethernet Header
|
| Source MAC
| Destination MAC (Router/Switch)
| EtherType
| CRC Trailer
|
| Ethernet Frame



| Layer 1 - Physical
| Converts Frame → Bits
|
| 1010101010101010
|
| Electrical / Optical / Radio Signal

NETWORK

Switch → Router → ISP



Google Server

(Reverse Process)

Bits



Frame



Packet



TCP Segment



Session



Presentation



Application

Google receives:

What Every Layer Actually Does

Layer	Job	Adds/Removes	Data Name
7 Application	Browser creates request	HTTP request	Data
6 Presentation	Encrypt, Compress, Encode	TLS/SSL processing	Data
5 Session	Open/Maintain/Close session	Session information	Data
4 Transport	Reliability, segmentation	TCP/UDP Header	Segment/Datagram
3 Network	Logical addressing & routing	IP Header	Packet
2 Data Link	Local delivery	MAC Header + CRC Trailer	Frame
1 Physical	Send bits over medium	Converts to electrical/light/radio signals	Bits

What is Added at Each Layer?

Application Layer

GET / HTTP/1.1

↓

Presentation Layer

TLS Encryption

↓

Session Layer

Session Information

↓

Transport Layer

```
+-----+
| TCP Header                                |
+-----+
| HTTP Request                             |
+-----+
```

↓

Network Layer

```
+-----+
| IP Header                                |
+-----+
| TCP Header                              |
+-----+
| HTTP Request                             |
+-----+
```

↓

Data Link Layer

```
+-----+
| Ethernet Header                          |
+-----+
| IP Header                                |
+-----+
| TCP Header                              |
+-----+
```

```
| HTTP Request |
+-----+
| Ethernet CRC Trailer |
+-----+

↓

Physical Layer

010101101011010110101...
```

Data Encapsulation (Nested Box Analogy)

```
+-----+
| Ethernet Frame |
|               |
| +-----+ |
| | IP Packet | | | | | |
| |         | |
| | +-----+ | |
| | | TCP Segment | | |
| | |         | | |
| | | +-----+ | | |
| | | | HTTP Request | | | |
| | | +-----+ | | |
| | +-----+ | |
| +-----+ |
+-----+
```

Each layer wraps the previous layer with its own control information.

What Happens During Reception?

The receiver **removes only its own header**.

```
Received Ethernet Frame

↓

Remove Ethernet Header

↓

IP Packet

↓
```

Remove IP Header

↓

TCP Segment

↓

Remove TCP Header

↓

Encrypted HTTP Request

↓

Decrypt

↓

Browser receives

GET / HTTP/1.1

This process is called **Decapsulation**.

Common Problems Mapped to OSI Layers

Layer 7 - Application

Problem: Browser cannot create HTTP request

Example: Browser crash, DNS API failure

|



Layer 6 - Presentation

Problem: HTTPS/TLS handshake fails

Example: Invalid certificate, TLS version mismatch

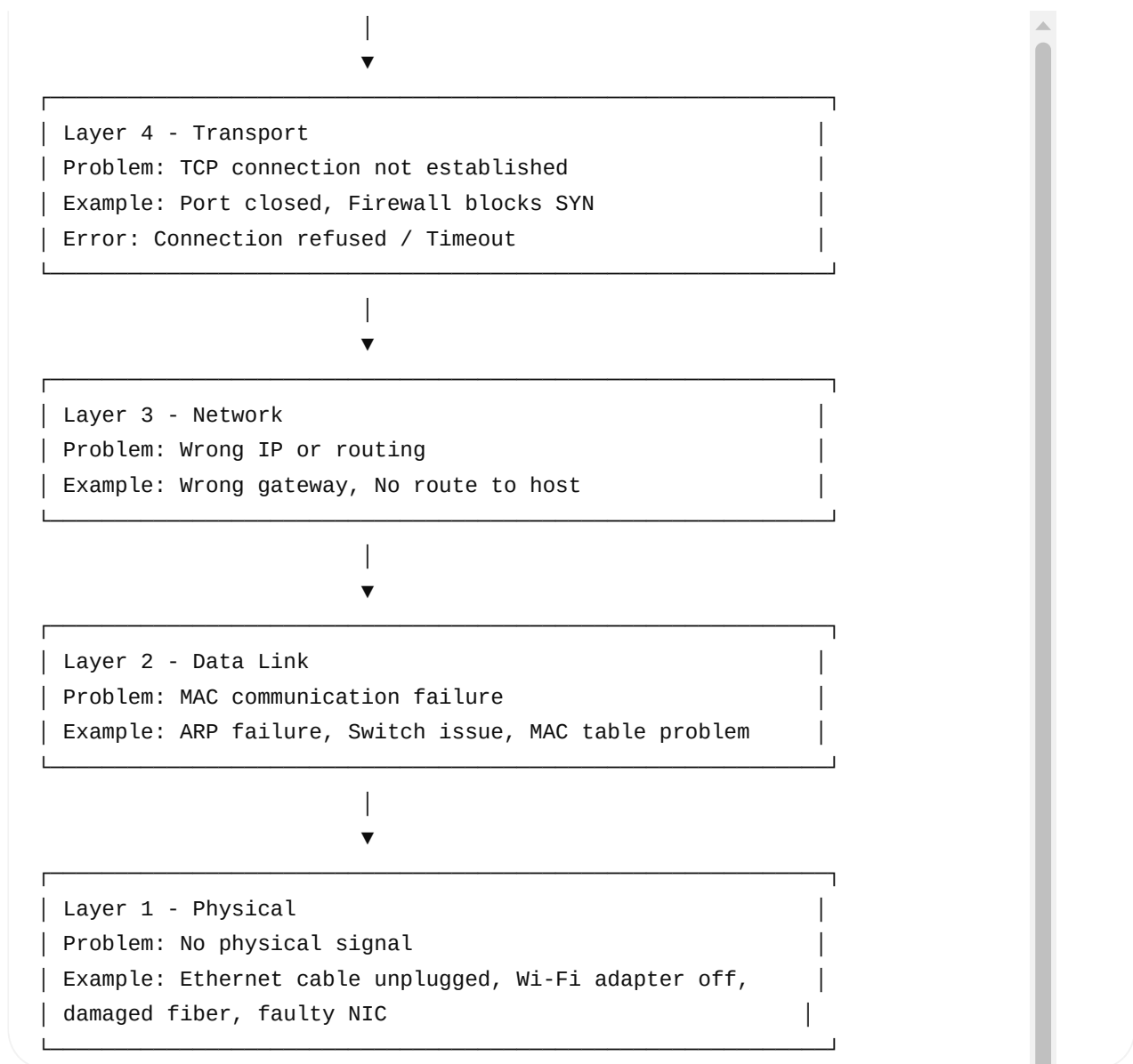
|



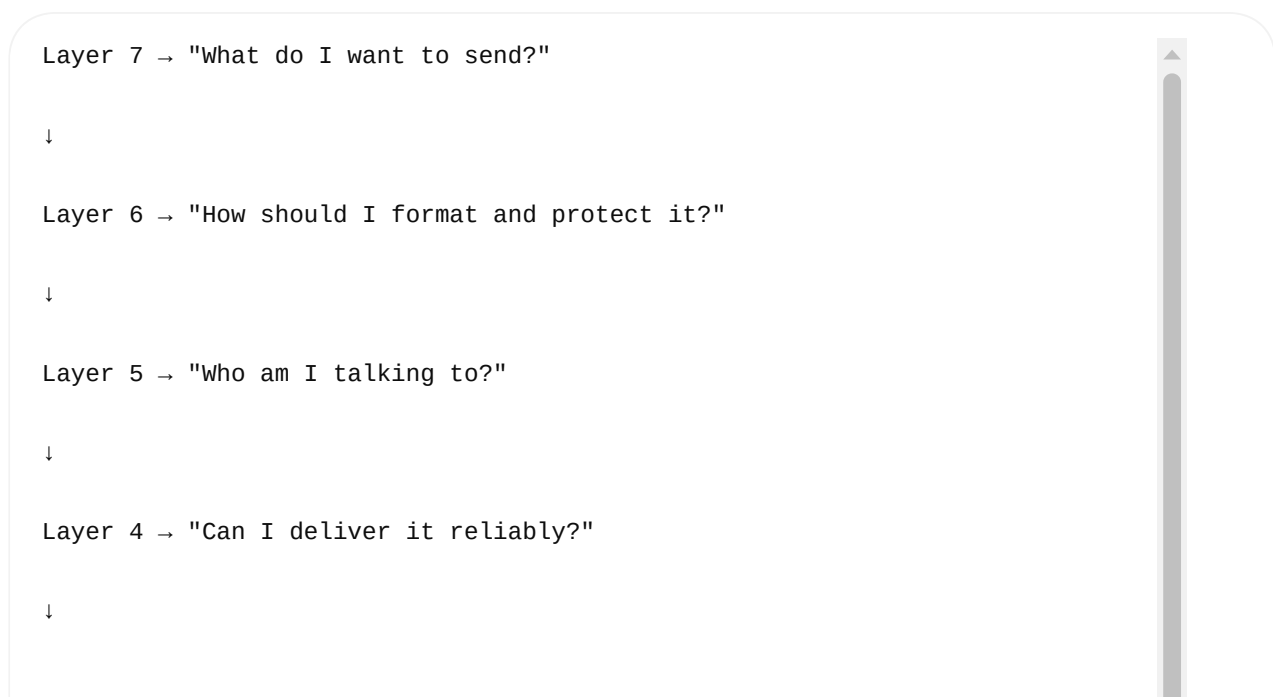
Layer 5 - Session

Problem: Session timeout

Example: Login expires, SSH session disconnects



Interview Memory Trick



Layer 3 → "Where should it go?"

↓

Layer 2 → "Who is the next device?"

↓

Layer 1 → "How do I physically transmit it?"

Linux Commands Related to Each Layer

OSI Layer	Linux Tools/Commands
Application	curl , wget , ssh , scp , ftp , dig
Presentation	openssl , gnutls-cli
Session	ssh , screen , tmux (session management examples)
Transport	ss , netstat , nc , telnet
Network	ip addr , ip route , ping , traceroute
Data Link	ip link , arp , ip neigh , ethtool
Physical	ethtool , lspci , lsusb , iwconfig (wireless)

Embedded Linux Interview Tip

For Embedded Linux roles, interviewers often ask **which OSI layers your application directly interacts with.**

- **C socket programming** mainly works at the **Transport layer (TCP/UDP)**, but your application logic runs at the **Application layer**.
- The **Linux kernel networking stack** automatically handles the **Network (IP)**, **Data Link (Ethernet)**, and **Physical** layers through device drivers and the network interface.
- As an application developer, you typically **do not manually create IP or Ethernet headers** when using standard sockets; the kernel constructs them for you. Understanding this separation is a common interview topic